Ruby Developer Interview Preparation Guide

Day 1: Core Ruby & Advanced Concepts

Theoretical Topics to Master

1. **Ruby Language Deep Dive**
    - **Module vs Class vs Mixin**: Understand the fundamental differences and use cases
    - **Method Lookup Chain**: How Ruby resolves method calls with inheritance, modules, and prepend
    - **Include vs Extend vs Prepend**: Implementation details and when to use each
    - **Metaprogramming**: **method_missing**, **define_method**, **instance_eval**, practical applications
    - **Blocks, Procs & Lambdas**: Differences in return behavior, arity checking, and closures
2. **Architecture & Design Principles**
    - **SOLID in Ruby**: Practical implementations of each principle
    - **Law of Demeter**: Implications for object coupling and message passing
    - **Service Objects & Query Objects**: Implementation patterns and best practices
    - **Microservices vs Monolith**: Trade-offs, communication patterns, deployment strategies
    - **Design Patterns**: Common patterns in Ruby (Observer, Decorator, Strategy, etc.)
3. **Database Knowledge**
    - **ACID Properties**: Atomicity, Consistency, Isolation, Durability in depth
    - **Transaction Management**: Isolation levels, deadlock prevention
    - **Advanced Indexing**: Composite indexes, partial indexes, expression indexes
    - **N+1 Query Problem**: Detection and solutions (includes, eager loading, bullet gem)

Practice Exercise 1: SOLID Refactoring

```
# Before refactoring - violates Single Responsibility and Open/Closed principles
class OrderProcessor
  def initialize(order)
    @order = order
  end

  def process
    if @order.type == "standard"
      # Process standard order
      calculate_standard_price
      send_standard_notification
    elsif @order.type == "express"
      # Process express order
      calculate_express_price
      send_express_notification
    elsif @order.type == "international"
      # Process international order
      calculate_international_price
      apply_customs_fee
      send_international_notification
    end

    save_order
  end

  private

  def calculate_standard_price
    @order.total = @order.items.sum(&:price) * 1.0
  end

  def calculate_express_price
    @order.total = @order.items.sum(&:price) * 1.2
```

```
  end

  def calculate_international_price
    @order.total = @order.items.sum(&:price) * 1.5
  end

  def apply_customs_fee
    @order.total += 25.0
  end

  def send_standard_notification
    # Send standard notification
    puts "Standard notification sent"
  end

  def send_express_notification
    # Send express notification
    puts "Express notification sent"
  end

  def send_international_notification
    # Send international notification
    puts "International notification sent"
  end

  def save_order
    # Save order to database
    puts "Order saved"
  end
end
```

Day 2: Rails, Testing & Advanced Topics

Theoretical Topics to Master

1. **Rails Deep Dive**
    - **Request-Response Cycle**: Complete flow from browser to database and back
    - **Rack Middleware**: Understanding the stack and custom middleware
    - **ActiveRecord Internals**: Query generation, caching mechanisms
    - **Polymorphic Associations**: Implementation and best practices
    - **Rails Engines**: Modularizing applications
2. **Testing Excellence**
    - **RSpec Best Practices**: Shared examples, contexts, custom matchers
    - **Test Doubles**: Mocks, stubs, and spies - when to use each
    - **Integration Testing**: System tests, request specs
    - **TDD/BDD Workflow**: Red-Green-Refactor in practice
3. **Performance & Optimization**
    - **Ruby GIL**: Understanding and working around limitations
    - **Memory Profiling**: Tools and techniques
    - **Background Processing**: Sidekiq, ActiveJob best practices
    - **Caching Strategies**: Fragment, Russian Doll, HTTP caching

Practice Exercise 2: Binary Search Implementation

Implement a binary search algorithm for a sorted array

```
def binary_search(array, target)
  # Your implementation here
end

# Test cases
puts binary_search([1, 2, 3, 4, 5], 3) # Should return 2 (index of target)
puts binary_search([1, 2, 3, 4, 5], 6) # Should return nil (target not found)
```

Practice Exercise 3: String Processing

```
# Implement a function that takes a string and replaces words longer than
# a specified length with a shortened version:
# Example: "internationalization" becomes "i18n" (first letter + count of middle letters + last letter)
def shorten_words(text, min_length = 10)
  # Your implementation here
end

# Test cases
puts shorten_words("The internationalization of our application is a complex task")
# Should output: "The i18n of our a10n is a c5x task"
```

Optimal Solutions

Solution for Exercise 1: SOLID Refactoring

```
# After refactoring - follows SOLID principles
```

```
# Base order processor with common functionality
class OrderProcessor
  def initialize(order)
    @order = order
  end

  def process
    calculate_price
    send_notification
    save_order
  end

  private

  def calculate_price
    raise NotImplementedError, "Subclasses must implement calculate_price"
  end

  def send_notification
    raise NotImplementedError, "Subclasses must implement send_notification"
  end

  def save_order
    # Common functionality
    puts "Order saved"
  end
end

# Specific processors for each order type
class StandardOrderProcessor < OrderProcessor
  private

  def calculate_price
    @order.total = @order.items.sum(&:price) * 1.0
```

```
  end

  def send_notification
    puts "Standard notification sent"
  end
end

class ExpressOrderProcessor < OrderProcessor
  private

  def calculate_price
    @order.total = @order.items.sum(&:price) * 1.2
  end

  def send_notification
    puts "Express notification sent"
  end
end

class InternationalOrderProcessor < OrderProcessor
  private

  def calculate_price
    @order.total = @order.items.sum(&:price) * 1.5
    apply_customs_fee
  end

  def apply_customs_fee
    @order.total += 25.0
  end

  def send_notification
    puts "International notification sent"
  end
end
```

```
# Factory to create the appropriate processor
class OrderProcessorFactory
  def self.for(order)
    case order.type
    when "standard" then StandardOrderProcessor.new(order)
    when "express" then ExpressOrderProcessor.new(order)
    when "international" then InternationalOrderProcessor.new(order)
    else
      raise ArgumentError, "Unknown order type: #{order.type}"
    end
  end
end

# Usage
processor = OrderProcessorFactory.for(order)
processor.process
```

Solution for Exercise 2: Binary Search

```
def binary_search(array, target)
  low = 0
  high = array.length - 1

  while low <= high
    mid = (low + high) / 2

    if array[mid] == target
      return mid
    elsif array[mid] < target
      low = mid + 1
    else
      high = mid - 1
    end
  end
```

```
  nil # Target not found
end
```

Solution for Exercise 3: String Processing

```
def shorten_words(text, min_length = 10)
  text.split.map do |word|
    if word.length >= min_length
      "#{word[0]}#{word.length - 2}#{word[-1]}"
    else
      word
    end
  end.join(' ')
end
```

Resources

1. **Ruby & Rails Documentation**
    - Ruby Documentation
    - Rails Guides
2. **Practice Platforms**
    - LeetCode Ruby Problems

- Exercism Ruby Track

3. **Advanced Reading**
    - Practical Object-Oriented Design in Ruby
    - Metaprogramming Ruby 2
    - Rails AntiPatterns